

Nextflow

W1: quality control and reporting on sequencing data

Instruction Only Prompt: We want to create a workflow using Nextflow that performs quality assessment of FASTQ files, preprocesses sequencing data, and generates an aggregated summary of the results. The input data should be provided via the sample sheet available at https://raw.githubusercontent.com/nf-core/test-datasets/viralrecon/samplesheet/samplesheet_test_illumina_amplicon.csv. Please outline the step-by-step modules required to build this workflow.

Response from gpt-4o:

To create a Nextflow workflow for **quality assessment, preprocessing of FASTQ files, and aggregated summary reporting**, using the sample sheet provided at [this link](#), you can follow this modular breakdown:

Workflow Overview

- **Input:** Sample sheet specifying paired/single-end FASTQ files
- **Steps:**
 1. Parse the sample sheet
 2. Perform initial quality control (FastQC)
 3. Trim adapters and low-quality bases
 4. Re-run quality control (FastQC)
 5. Generate a consolidated MultiQC report

Step-by-Step Modules

Module 1: Parse Input Sample Sheet

- **Purpose:** Read and interpret CSV to locate FASTQ file paths.
- **Tool:** `csvtk`, `awk`, or native Nextflow parsing (`Channel.fromPath + splitCsv`)
- **Output:** Channel of sample identifiers and corresponding FASTQ paths.

Module 2: Initial Quality Control

- **Purpose:** Evaluate quality metrics of raw reads.
- **Tool:** `FastQC`
- **Input:** Raw FASTQ files
- **Output:** `.html` and `.zip` reports per sample

Module 3: Adapter and Quality Trimming

- **Purpose:** Remove adapters and trim low-quality bases.
- **Tools (choose one):** `Trim Galore!`, `fastp`, or `Trimmomatic`
- **Input:** Raw FASTQ files
- **Output:** Cleaned FASTQ files

Module 4: Post-trim Quality Control

- **Purpose:** Reassess quality after trimming.
- **Tool:** `FastQC`
- **Input:** Cleaned FASTQ files
- **Output:** Post-trim `.html` and `.zip` reports

Module 5: Aggregated Summary Report

- **Purpose:** Compile all FastQC results into a single report.

- **Tool:** MultiQC
 - **Input:** All FastQC outputs (raw and trimmed)
 - **Output:** multiqc_report.html
-

Nextflow Execution Flow

```
workflow {  
  
    samplesheet_ch = Channel.fromPath("samplesheet_test_illumina_amplicon.csv")  
        .splitCsv(header:true)  
  
    parsed_samples = parseSamples(samplesheet_ch)  
  
    raw_qc = fastqc(parsed_samples)  
    trimmed = trim_reads(parsed_samples)  
    post_qc = fastqc(trimmed)  
  
    multiqc(raw_qc.mix(post_qc))  
}
```

Each process (`parseSamples`, `fastqc`, `trim_reads`, `multiqc`) can be implemented as a separate `process {}` block in Nextflow using containerized tools via Docker/Singularity/Conda.

Output Directory Structure (Example)

results/

```
|— fastqc/
| |— raw/
| |— trimmed/
|— trimmed_fastq/
|— multiqc/
| |— multiqc_report.html
```

Let me know if you'd like a full Nextflow script implementation or a DSL2-compatible module structure!

Response from Gemini 2.5 Flash:

To build a Nextflow workflow that performs quality assessment, preprocessing, and generates an aggregated summary of FASTQ files, you would typically use a combination of standard bioinformatics tools encapsulated within Nextflow processes, leveraging the modularity encouraged by nf-core/modules.

Here's a step-by-step outline of the modules required for this workflow, assuming an nf-core-style structure:

Workflow Name: nf-core/fastq_qc (or similar)

Input Data: The workflow will take the provided sample sheet https://raw.githubusercontent.com/nf-core/test-datasets/viralrecon/samplesheet/samplesheet_test_illumina_amplicon.csv as its primary input, which points to the FASTQ files.

Modules/Processes Required:

1. **Input Parsing (read_params_and_samplesheet / get_fastq_channels)**
 - **Purpose:** Reads the input samplesheet (CSV) and parses it to create structured channels of FASTQ file paths, typically grouped by sample and pair (R1/R2).
 - **Description:** This is usually a core Nextflow script logic, often involving `Channel.fromPath` and `Channel.fromFilePairs` to identify and group the raw sequencing data. It ensures that downstream processes receive the correct input files per sample.

2. Initial Quality Control (FASTQC)

- **Module Name (example):** NFCORE_MODULES:FASTQC
- **Purpose:** Assesses the quality of the *raw*, unprocessed FASTQ files.
- **Input:** Raw FASTQ file channels (e.g., `ch_fastqs_raw`).
- **Output:** FASTQC HTML report and data file for each raw FASTQ file.
- **Tool:** FASTQC

3. Read Preprocessing / Trimming (SEQTK_TRIM or TRIMGALORE)

- **Module Name (example):** NFCORE_MODULES:SEQTK:TRIM or NFCORE_MODULES:TRIMGALORE
- **Purpose:** Removes adapter sequences and low-quality bases from the raw FASTQ files. This is a crucial preprocessing step for most sequencing data.
- **Input:** Raw FASTQ file channels (e.g., `ch_fastqs_raw`).
- **Output:** Trimmed FASTQ file channels (e.g., `ch_fastqs_trimmed`).
- **Tool:** `seqtk trimfq` (as seen in `nf-core/demo`) or Trim Galore! (which uses Cutadapt). Trim Galore! is often preferred for more comprehensive adapter detection and trimming.

4. Post-Preprocessing Quality Control (FASTQC)

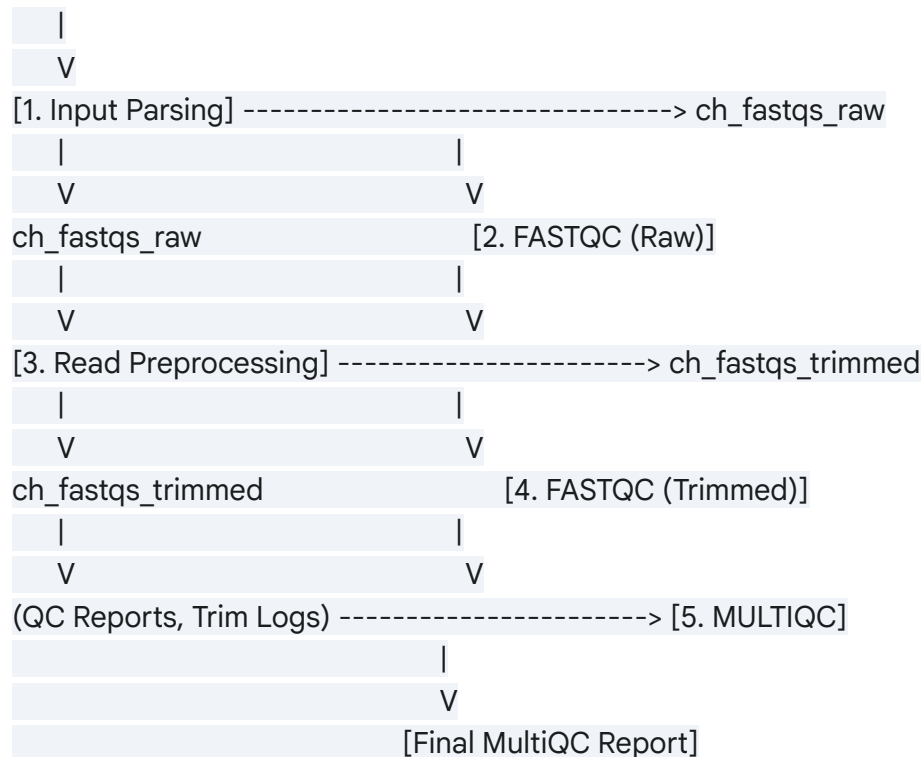
- **Module Name (example):** NFCORE_MODULES:FASTQC (re-use of the same module)
- **Purpose:** Assesses the quality of the *trimmed* FASTQ files to evaluate the effectiveness of the preprocessing step.
- **Input:** Trimmed FASTQ file channels (e.g., `ch_fastqs_trimmed`).
- **Output:** FASTQC HTML report and data file for each trimmed FASTQ file.
- **Tool:** FASTQC

5. Aggregate Quality Summary (MULTIQC)

- **Module Name (example):** NFCORE_MODULES:MULTIQC
- **Purpose:** Gathers all the quality control reports (from both raw and trimmed FASTQC runs, and potentially trimming logs) and generates a single, comprehensive, interactive HTML report.
- **Input:** All relevant output files from previous QC and preprocessing steps (e.g., FASTQC reports, trimming logs).
- **Output:** A single MultiQC HTML report (`multiqc_report.html`) and associated data files.
- **Tool:** MultiQC

Workflow Flow (Conceptual Diagram):

[samplesheet.csv]



This modular approach allows for easy maintenance, updates, and reusability of individual tools within different Nextflow pipelines, following nf-core's best practices.

Response from DeepSeek-V3:

Here's a step-by-step outline for building a Nextflow workflow for FASTQ quality assessment and preprocessing, using nf-core modules and best practices:

Workflow Overview

1. Input: Samplesheet with FASTQ paths
2. Steps:
 - Quality Control (FastQC)
 - Adapter/Quality Trimming (Cutadapt)
 - Read Filtering (FastP)
 - Post-trimming QC (FastQC)
 - Aggregated Report (MultiQC)

3. Output: Processed reads + comprehensive QC report

Step-by-Step Modules & Implementation

1. Input Specification

- Use the provided samplesheet ([link](#)) with columns:

```
sample,fastq_1,fastq_2  # for paired-end data
```

2. Required nf-core Modules

Add these to your `modules.config` and import in `main.nf`:

Module	Purpose	nf-core Module Link
<code>fastqc</code>	Raw read QC	nf-core/fastqc
<code>cutadapt</code>	Adapter trimming	nf-core/cutadapt
<code>fastp</code>	Quality filtering	nf-core/fastp
<code>multiqc</code>	Aggregate reports	nf-core/multiqc

3. Workflow Design (`main.nf`)

```
nextflow
include { FASTQC } from '../modules/nf-core/fastqc/main'
include { CUTADAPT } from '../modules/nf-core/cutadapt/main'
include { FASTP } from '../modules/nf-core/fastp/main'
include { MULTIQC } from '../modules/nf-core/multiqc/main'
```

```

workflow {
  // Step 1: Read samplesheet
  Channel.fromPath(params.input)
    | splitCsv(header: true)
    | map { row -> tuple(row.sample, [file(row.fastq_1), file(row.fastq_2)]) }
    | set { raw_reads }

  // Step 2: Initial QC
  FASTQC(raw_reads)

  // Step 3: Adapter trimming
  CUTADAPT(raw_reads, params.adapters)

  // Step 4: Quality filtering
  FASTP(CUTADAPT.out.trimmed)

  // Step 5: Post-trimming QC
  FASTQC(FASTP.out.cleaned)

  // Step 6: Aggregate reports
  MULTIQC(
    fastqc: FASTQC.out.zip.collect(),
    cutadapt: CUTADAPT.out.log.collect(),
    fastp: FASTP.out.json.collect(),
    prefix: "QC_Summary"
  )
}

```

4. Configuration (nextflow.config)

```

nextflow

params {
  input = "samplesheet.csv"
  adapters = "file:adapters.fa" // Illumina adapters
  outdir = "results"
}

profiles {
  docker { docker.enabled = true }
  singularity { singularity.enabled = true }
}

```

5. Run the Workflow


```
nextflow run main.nf -profile docker --input samplesheet.csv
```

Expected Output Structure

text

```
results/
├─ fastqc/           # Initial QC reports
├─ cutadapt/         # Trimmed FASTQs + logs
├─ fastp/            # Filtered FASTQs + JSON
├─ post_trim_fastqc/ # Post-processing QC
└─ multiqc/          # Final HTML summary
```

Key Customizations

1. Trim Parameters: Adjust `cutadapt/fastp` args in `conf/modules.config`.
2. Additional Steps: Add modules like `bowtie2` for alignment if needed.
3. Samplesheet Handling: Validate input with a custom `check_samplesheet.py` script.

For a production-ready pipeline, refer to [nf-core/rnaseq](#) as a template. Would you like help with any specific module implementation?

[W2: nf-core/fetchngs](#): A bioinformatics pipeline to fetch metadata and raw FastQ files from databases.

Response from GPT-4o:

Here is a **step-by-step breakdown** of modules to create a **Nextflow workflow** that takes a CSV of sample IDs (from SRA, ENA, GEO, or DDBJ), retrieves metadata, and downloads associated FASTQ files:

Workflow Objective

Build a modular, reproducible Nextflow pipeline that automates the process of:

- Parsing a user-provided CSV file of sample accessions
 - Retrieving sample metadata
 - Downloading raw FASTQ files
 - Organizing output files
-

Step-by-Step Modules

♦ 1. Read CSV Input

- **Input:** A CSV file with one column (`sample_id`) of accession IDs.
- **Implementation:**
 - Use `Channel.fromPath()` and `.splitCsv()` in Nextflow
 - Emit one ID per channel item

Channel

```
.fromPath('samples.csv')  
.splitCsv(header: true)  
.map { row -> row.sample_id }
```

♦ 2. Retrieve Metadata

- **Purpose:** For each ID, collect metadata such as experiment design, layout (single/paired), platform, etc.
- **Tool Choices:**
 - [enaBrowserTools](#) – `enaDataGet` (ENA/DDBJ)

- [pysradb](#) – SRA/GEO metadata
 - [Entrez Direct](#) – NCBI SRA/GEO
 - Custom Python scripts with Biopython or pandas
 - **Output:** JSON or TSV metadata file per sample.
-

◆ 3. Download FASTQ Files

- **Purpose:** Download raw reads from corresponding repositories.
 - **Tool Choices:**
 - [fasterq-dump](#) (SRA)
 - [ascp](#) (Aspera) or [wget/curl](#) (ENA/DDBJ)
 - [enaDataGet](#) (FTP or Aspera, ENA/DDBJ)
 - nf-core [fetchngs](#) modules for automation
 - **Considerations:**
 - Handle paired-end vs single-end layout
 - Retry on failure
 - Store in structured directory: [results/fastq/{sample_id}/](#)
-

◆ 4. Validate and Organize Outputs

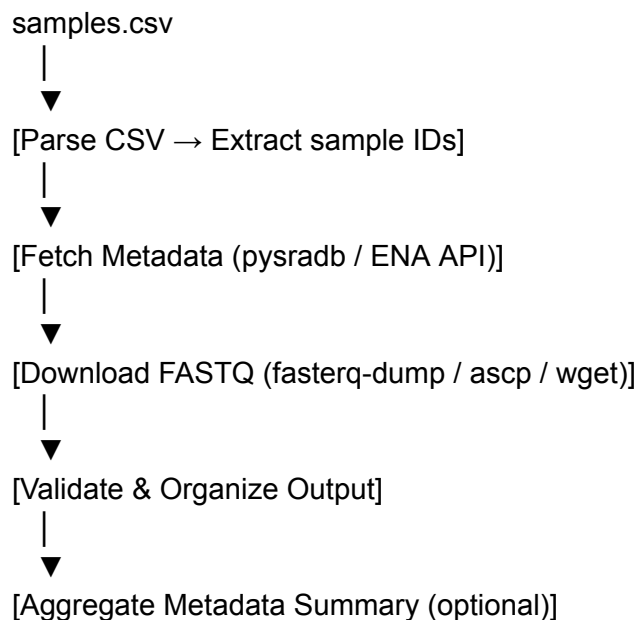
- **Purpose:** Verify download success, log failures, organize into directories.
- **Tasks:**

- Check if FASTQ files exist
- Move files to per-sample folders
- Optionally validate with `seqkit stats` or `FastQC`

◆ 5. Aggregate Metadata (Optional)

- **Purpose:** Merge individual metadata files into one summary.
- **Tools:**
 - `csvkit`, `pandas`, or R (`dplyr`)
- **Output:** `results/metadata_summary.csv`

Workflow Summary Diagram





Optional Enhancements

- Add `params.database` to specify SRA/ENA/GEO
 - Containerize tools with `conda` or `docker`
 - Log failed downloads
 - Integrate with `MultiQC` for QC summary
-

Would you like a scaffolded `main.nf` + `nextflow.config` to get started with implementation?

Response from Gemini 2.5 Flash:

Here's a step-by-step outline for creating a Nextflow workflow to fetch NGS data (metadata and FASTQ files) from public databases, based on the functionalities of nf-core/fetchngs.

Workflow Name: `ngs_data_fetcher`

Input: A CSV file (`--input_csv`) containing sample database IDs (e.g., SRA, ENA, GEO, or DDBJ), typically with one ID per row.

Output:

- Downloaded FASTQ files, organized by run ID.
 - A consolidated `samplesheet.csv` containing metadata and paths to the downloaded FASTQ files, ready for downstream analysis.
-

Workflow Structure:

We will organize the workflow using Nextflow's DSL2, separating the main workflow logic, configuration, and individual processes (modules).

1. **main.nf:** The main workflow script, defining parameters and orchestrating the execution of modules.
2. **conf/modules.config:** Configuration file for process resources, container images, and general parameters.
3. **modules/ directory:** Contains individual `.nf` files for each distinct processing step

(Nextflow processes).

Step-by-Step Modules Outline:

1. main.nf - Main Workflow Definition

This file will define the overall workflow, including its parameters, input/output channels, and the sequence of module calls.

Code snippet

```
// main.nf
nextflow.enable.dsl2

// Import individual modules
include { PARSE_INPUT_IDS } from './modules/parse_input_ids'
include { FETCH_METADATA } from './modules/fetch_metadata'
include { DOWNLOAD_FASTQ_DIRECT } from './modules/download_fastq_direct'
include { DOWNLOAD_FASTQ_SRA } from './modules/download_fastq_sra'
include { VALIDATE_MD5 } from './modules/validate_md5'
include { COLLATE_SAMPLESHEET } from './modules/collate_samplesheet'

// Define workflow parameters
params {
    input_csv = false // Path to the input CSV file
    outdir = 'results' // Output directory
    ena_api_base_url = 'https://www.ebi.ac.uk/ena/portal/api/filereport'
    // Add other relevant parameters like 'column_name_for_ids' if your CSV header is variable
}

workflow ngs_data_fetcher {
```

```

// Check for required input
if (!params.input_csv) {
    exit 1, "Please provide an input CSV file using --input_csv"
}

// Define input channel from the CSV file
// Using fromPath to automatically create a channel from the file
Channel
.fromPath(params.input_csv)
.set { ch_input_csv }

main:
    // 1. Parse Input IDs
    // Takes the input CSV and extracts a list of database IDs.
    PARSE_INPUT_IDS(ch_input_csv)

    // 2. Fetch Metadata
    // Uses the IDs to query the ENA API and retrieve comprehensive metadata.
    FETCH_METADATA(PARSE_INPUT_IDS.out.ids_list)

    // 3. Download FASTQ (Direct URLs)
    // Attempts to download FASTQ files directly using wget or curl from ENA-provided URLs.
    DOWNLOAD_FASTQ_DIRECT(FETCH_METADATA.out.metadata_by_run)

    // 4. Download FASTQ (SRA Tools Fallback)
    // If direct download links are not available or fail, use sra-tools (prefetch, fastq-dump)
    // This process would typically receive *only* the IDs that failed direct download.
    // For simplicity in this outline, it receives all metadata and internally decides.
    DOWNLOAD_FASTQ_SRA(FETCH_METADATA.out.metadata_by_run)

    // Combine results from both download methods.
    // This needs careful merging to ensure each run has its FASTQs, prioritizing direct
    downloads.
    // A more robust approach would identify which runs were handled by which process.
    ch_all_downloaded_fastqs = DOWNLOAD_FASTQ_DIRECT.out.downloaded_fastqs.mix(
        DOWNLOAD_FASTQ_SRA.out.downloaded_fastqs)

    // 5. Validate MD5 Checksums
    // Verifies the integrity of downloaded FASTQ files against provided MD5 sums.
    VALIDATE_MD5(ch_all_downloaded_fastqs, FETCH_METADATA.out.metadata_by_run)

```

```

    // 6. Collate Samplesheet
    // Combines all fetched metadata and paths to validated FASTQ files into a final
samplesheet.
    COLLATE_SAMPLESHEET(FETCH_METADATA.out.metadata_by_run,
VALIDATE_MD5.out.validated_fastqs)

emit:
    // Expose the final samplesheet and downloaded files as workflow outputs
    samplesheet = COLLATE_SAMPLESHEET.out.final_samplesheet
    downloaded_files = VALIDATE_MD5.out.validated_fastqs
}

```

2. conf/modules.config - Configuration File

This file centralizes resource requests and container definitions for each process, ensuring reproducibility and efficient execution across different environments.

Code snippet

```

// conf/modules.config
params {
    // Default ENA API URL
    ena_api_base_url = 'https://www.ebi.ac.uk/ena/portal/api/filereport'

    // General resource defaults
    max_cpus = 4
    max_memory = '16GB'
    max_time = '240h'
}

process {
    // Default container for processes without specific overrides.
    // Ensure this container includes common tools like Python, curl, wget, etc.
    container = 'quay.io/biocontainers/python:3.9.7--h57b4202_1' // Python for scripting
}

```



```

// Process-specific resource and container overrides
withName: PARSE_INPUT_IDS {
  cpus = 1
  memory = '1GB'
  container = 'quay.io/biocontainers/coreutils:8.32--h11921a8_0' // For cut/awk, or can use
default Python
}
withName: FETCH_METADATA {
  cpus = 1
  memory = '2GB'
  // Python with 'requests' library for API calls
  container = 'quay.io/biocontainers/python-requests:2.28.1--pyhd8ed1ab_0'
}
withName: DOWNLOAD_FASTQ_DIRECT {
  cpus = 1
  memory = '4GB'
  time = '24h' // Downloads can take long
  container = 'quay.io/biocontainers/wget:1.21.3--h400030e_0' // Includes wget and curl
}
withName: DOWNLOAD_FASTQ_SRA {
  cpus = 2 // sra-tools can sometimes benefit from more cores
  memory = '8GB'
  time = '48h' // Prefetch and fastq-dump can be time-consuming for large files
  container = 'quay.io/biocontainers/sra-tools:3.0.0--h9ee0642_0' // Contains prefetch and
fastq-dump
}
withName: VALIDATE_MD5 {
  cpus = 1
  memory = '1GB'
  container = 'quay.io/biocontainers/coreutils:8.32--h11921a8_0' // For md5sum
}
withName: COLLATE_SAMPLESHEET {
  cpus = 1
  memory = '2GB'
  container = 'quay.io/biocontainers/pandas:1.5.0--py39he8283a0_0' // For Python with
pandas
}
}

```

3. modules/ Directory - Individual Process Definitions

Each of these will be a separate .nf file within the modules/ directory.

3.1. modules/parse_input_ids.nf

- **Purpose:** Reads the input CSV file and extracts a list of database IDs. It should handle headers and extract from a specified column.
- **Input:** path input_csv
- **Output:** path ids_list (a file containing one ID per line)
- **Script:** Uses basic shell commands (tail, cut) or a Python script for robust CSV parsing.

Code snippet

```
// modules/parse_input_ids.nf
process PARSE_INPUT_IDS {
  tag "Parse IDs from ${input_csv.baseName}"
  input:
    path input_csv
  output:
    path "ids.txt", emit: ids_list
  script:
    """
    # Assuming IDs are in the first column and there's a header line to skip.
    # For a more robust solution, use a Python script to specify column by name
    # e.g., using pandas.
    tail -n +2 "${input_csv}" | cut -d',' -f1 > ids.txt
    """
}
```

3.2. modules/fetch_metadata.nf

- **Purpose:** Queries the ENA API using the extracted IDs to retrieve detailed metadata (e.g., run accessions, FTP links, MD5 checksums).
- **Input:** path ids_list
- **Output:** path metadata_by_run (e.g., a JSON or TSV file containing metadata for each run ID)
- **Script:** A Python script utilizing the requests library to interact with the ENA API, parsing the response.

Code snippet

```
// modules/fetch_metadata.nf
process FETCH_METADATA {
    tag "Fetch ENA metadata"
    input:
        path ids_list
    output:
        path "metadata_by_run.json", emit: metadata_by_run
    script:
        """
        python <<EOF
        import requests
        import json
        import sys

        ids = [line.strip() for line in open('$ids_list')]
        # Base URL for ENA API file report (can be passed via params)
        ena_api_url = '$params.ena_api_base_url'

        all_metadata = []
        # Batch IDs for API query to avoid URL length limits and be efficient
        batch_size = 200
        for i in range(0, len(ids), batch_size):
            batch_ids = ids[i:i + batch_size]
            accession_list = ','.join(batch_ids)

            params = {
                'accession': accession_list,
                'result': 'read_run',
```

```

        'fields':
'run_accession,study_accession,sample_accession,experiment_accession,fastq_ftp,submitted
_ftp,sra_ftp,md5,library_strategy,library_source,library_selection,library_layout,platform,instru
ment_model',
        'format': 'json'
    }
    try:
        response = requests.get(ena_api_url, params=params)
        response.raise_for_status() # Raise an exception for HTTP errors
        all_metadata.extend(response.json())
    except requests.exceptions.RequestException as e:
        print(f"Error fetching metadata for batch: {e}", file=sys.stderr)
        # Decide how to handle errors: skip batch, exit, log and continue

    with open('metadata_by_run.json', 'w') as f:
        json.dump(all_metadata, f, indent=4)
EOF
"""
}

```

3.3. modules/download_fastq_direct.nf

- **Purpose:** Attempts to download FASTQ files using direct FTP/HTTP links provided in the metadata (e.g., from fastq_ftp field).
- **Input:** path metadata_file (from FETCH_METADATA)
- **Output:** path downloaded_fastqs_list (a file listing paths to successfully downloaded files)
- **Script:** Parses the metadata_file to extract URLs and uses wget or curl for download.

Code snippet

```

// modules/download_fastq_direct.nf
process DOWNLOAD_FASTQ_DIRECT {
    tag "Download FASTQ direct"
    input:

```

```

    path metadata_file
output:
    path "downloaded_direct_fastqs.txt", emit: downloaded_fastqs
script:
    """
python <<EOF
import json
import os
import subprocess
import sys

with open('$metadata_file', 'r') as f:
    metadata = json.load(f)

downloaded_files = []
for entry in metadata:
    run_acc = entry.get('run_accession')
    fastq_ftps = entry.get('fastq_ftp', "").split(';') # Can be multiple comma-separated
    if not fastq_ftps or all(not url.strip() for url in fastq_ftps):
        # No direct FASTQ FTP links available for this entry
        continue

    for url in fastq_ftps:
        url = url.strip()
        if not url: continue

        filename = os.path.basename(url)
        # Create a directory for each run accession to organize files
        os.makedirs(run_acc, exist_ok=True)
        output_path = os.path.join(run_acc, filename)

        print(f"Attempting direct download: {url} to {output_path}", file=sys.stderr)
        try:
            # Use wget for downloading
            subprocess.check_call(['wget', '-q', '--show-progress', '-O', output_path, url])
            if os.path.exists(output_path) and os.path.getsize(output_path) > 0:
                downloaded_files.append(output_path)
            else:
                print(f"Warning: Downloaded file {output_path} is empty or missing.",
file=sys.stderr)
        except subprocess.CalledProcessError as e:

```

```

        print(f"Error downloading {url} with wget: {e}", file=sys.stderr)
    except Exception as e:
        print(f"An unexpected error occurred during wget download of {url}: {e}",
              file=sys.stderr)

    with open('downloaded_direct_fastqs.txt', 'w') as f:
        for fq_path in downloaded_files:
            f.write(fq_path + '\n')
    EOF
    """
}

```

3.4. modules/download_fastq_sra.nf

- **Purpose:** Serves as a fallback for downloads. If direct links are unavailable or fail, this module uses sra-tools (prefetch to download .sra files and fastq-dump to convert them to FASTQ).
- **Input:** path metadata_file (from FETCH_METADATA)
- **Output:** path downloaded_fastqs_list (a file listing paths to successfully downloaded files)
- **Script:** Extracts SRA accessions, uses prefetch and fastq-dump.

Code snippet

```

// modules/download_fastq_sra.nf
process DOWNLOAD_FASTQ_SRA {
    tag "Download FASTQ SRA tools"
    input:
        path metadata_file
    output:
        path "downloaded_sra_fastqs.txt", emit: downloaded_fastqs
    script:
        """
        python <<EOF
        import json

```

```

import os
import subprocess
import sys

with open('$metadata_file', 'r') as f:
    metadata = json.load(f)

downloaded_files = []
for entry in metadata:
    run_acc = entry.get('run_accession')
    # Check if direct fastq links exist; if so, skip SRA fallback for this run
    fastq_ftps = entry.get('fastq ftp', '').split(';')
    if fastq_ftps and any(url.strip() for url in fastq_ftps):
        print(f"Skipping SRA download for {run_acc} (direct links found).", file=sys.stderr)
        continue

    if not run_acc: continue # Skip if no run accession

    # Create a directory for each run accession to organize files
    os.makedirs(run_acc, exist_ok=True)
    # Change to the run's directory for prefetch/fastq-dump
    os.chdir(run_acc)

    print(f"Attempting SRA download for {run_acc} using prefetch and fastq-dump",
file=sys.stderr)
    try:
        # 1. Prefetch SRA file
        # prefetch will place the .sra file in a directory structure within the current dir
        subprocess.check_call(['prefetch', run_acc])

        # Find the path to the downloaded SRA file (can be tricky, depends on sra-tools config)
        # A common pattern: SRR/SRRXXXXXXX/SRRXXXXXXX.sra or just SRRXXXXXXX.sra in
work dir
        sra_file_path = os.path.join(run_acc, run_acc + '.sra') # common case
        if not os.path.exists(sra_file_path):
            # Fallback to finding it if prefetch puts it directly in current dir or another subdir
            # This part is highly dependent on sra-tools behavior.
            # nf-core/fetchngs uses `vdb-config` to set the output directory.
            print(f"Could not find SRA file at {sra_file_path}. Searching current directory.",
file=sys.stderr)
            found_sra = False

```

```

        for root, dirs, files in os.walk('.'):
            for file in files:
                if file.endswith('.sra') and file.startswith(run_acc):
                    sra_file_path = os.path.join(root, file)
                    found_sra = True
                    break
            if found_sra: break
        if not found_sra:
            raise FileNotFoundError(f"SRA file for {run_acc} not found after prefetch.")

# 2. fastq-dump
# --split-files for paired-end, --gzip for compression
subprocess.check_call(['fastq-dump', '--split-files', '--gzip', sra_file_path])

# Collect generated FASTQ files (e.g., SRR_1.fastq.gz, SRR_2.fastq.gz or SRR.fastq.gz)
for fq_suffix in ['_1.fastq.gz', '_2.fastq.gz', '.fastq.gz']:
    fq_filename = f"{run_acc}{fq_suffix}"
    if os.path.exists(fq_filename) and os.path.getsize(fq_filename) > 0:
        downloaded_files.append(os.path.join(run_acc, fq_filename)) # Store full path
    elif fq_suffix == '.fastq.gz' and not os.path.exists(f"{run_acc}_1.fastq.gz"):
        # For single-end reads if _1.fastq.gz doesn't exist
        pass # Already checked for _1 and _2, now check for single file
    elif fq_suffix == '_2.fastq.gz' and not os.path.exists(f"{run_acc}_1.fastq.gz"):
        # If it's a paired-end run, but _1 wasn't found, then this isn't right
        pass

except subprocess.CalledProcessError as e:
    print(f"Error with SRA tools for {run_acc}: {e}", file=sys.stderr)
except FileNotFoundError as e:
    print(f"Error: {e}", file=sys.stderr)
except Exception as e:
    print(f"An unexpected error occurred during SRA tools process for {run_acc}: {e}",
file=sys.stderr)
finally:
    os.chdir('.') # Go back to original directory

with open('../downloaded_sra_fastqs.txt', 'w') as f: # Write to parent directory
    for fq_path in downloaded_files:
        f.write(fq_path + '\n')

```



```

EOF
    """
}

```

3.5. modules/validate_md5.nf

- **Purpose:** Validates the integrity of downloaded FASTQ files by comparing their computed MD5 checksums with those provided in the ENA metadata.
- **Input:** path downloaded_fastqs_list, path metadata_file
- **Output:** path validated_fastqs_list (a file listing paths to validated FASTQ files)
- **Script:** Uses md5sum and a Python script to compare.

Code snippet

```

// modules/validate_md5.nf
process VALIDATE_MD5 {
    tag "Validate MD5 checksums"
    input:
        path downloaded_fastqs_list
        path metadata_file
    output:
        path "validated_fastqs.txt", emit: validated_fastqs
    script:
        """
        python <<EOF
        import json
        import os
        import subprocess
        import sys

        with open('$metadata_file', 'r') as f:
            metadata = json.load(f)

        # Create a mapping of filename (or run_accession + filename part) to expected MD5
        expected_md5s_map = {}

```

```

for entry in metadata:
    run_acc = entry.get('run_accession')
    md5_values = entry.get('md5', '').split(';')
    fastq_ftps = entry.get('fastq_ftp', '').split(';') # For deriving filename from original URL

    # Map each MD5 to its corresponding expected filename (based on how files are named
    # post-download)
    # This mapping logic needs to be robust to handle single/paired-end and naming
    # conventions
    # For simplicity, let's assume we map original URLs to their MD5s
    for i, url in enumerate(fastq_ftps):
        if url and i < len(md5_values):
            filename = os.path.basename(url.strip())
            expected_md5s_map[filename] = md5_values[i].strip()

    # Also consider SRA-tool naming: SRRxxxxxx_1.fastq.gz, SRRxxxxxx.fastq.gz
    if run_acc:
        for suffix in ['_1.fastq.gz', '_2.fastq.gz', '.fastq.gz']:
            filename_sra = f"{run_acc}{suffix}"
            # This part is tricky: ENA MD5s are for original files, not fastq-dumped ones directly.
            # Often the 'md5' field in ENA API refers to the SRA file's MD5 or a combined FASTQ
            # file.
            # A robust solution might need to check the SRA file's MD5, or trust fastq-dump.
            # For this exercise, we'll try to match against filename, knowing it's an
            # approximation.
            # If the md5 in ENA is for the SRA file itself, this check won't work for extracted
            # FASTQs.
            # For a real pipeline, it's safer to MD5 check the SRA file, or use ENA's file report
            # md5s accurately.
            if filename_sra in expected_md5s_map: # If the original FTP filename matches
            # SRA-dumped
                pass # Use that MD5
            else: # Placeholder: in reality, need to correctly associate ENA MD5 with derived
            # FASTQ names
                # For simplicity, assign SRA accessions to a generic MD5 check.
                # A proper solution might involve parsing ENA's 'checksum' field for the specific
            # files.
                if 'md5' in entry and entry['md5'] and run_acc in entry['md5']: # Basic check
                    expected_md5s_map[filename_sra] = entry['md5'].split(';')[0].strip() # Use first
            # MD5

```

```

validated_files = []
with open('$downloaded_fastqs_list', 'r') as f:
    downloaded_paths = [line.strip() for line in f]

for fq_path in downloaded_paths:
    fq_filename = os.path.basename(fq_path)
    # Need to adjust fq_filename to match key in expected_md5s_map based on how files
were named
    # For files downloaded directly, fq_filename matches original.
    # For SRA-dumped, it's like SRRXXXXXX_1.fastq.gz.
    # This mapping is the critical point here.
    # A simple strategy: remove _1.fastq.gz, _2.fastq.gz to get base name
(SRRxxxxxx.fastq.gz)
    base_filename_for_md5_lookup = fq_filename.replace('_1.fastq.gz',
'.fastq.gz').replace('_2.fastq.gz', '.fastq.gz')

    expected_md5 = expected_md5s_map.get(base_filename_for_md5_lookup) or
expected_md5s_map.get(fq_filename)

    if not expected_md5:
        print(f"Warning: No expected MD5 found for {fq_path}. Skipping validation.",
file=sys.stderr)
        validated_files.append(fq_path) # Assume valid if no check possible
        continue

    try:
        calculated_md5 = subprocess.check_output(['md5sum', fq_path]).decode().split(' ')[0]
        if calculated_md5 == expected_md5:
            print(f"MD5 validated for {fq_path}", file=sys.stderr)
            validated_files.append(fq_path)
        else:
            print(f"MD5 MISMATCH for {fq_path}. Expected: {expected_md5}, Got:
{calculated_md5}", file=sys.stderr)
            # You might choose to skip adding this to validated_files or flag it
    except Exception as e:
        print(f"Error calculating MD5 for {fq_path}: {e}", file=sys.stderr)

with open('validated_fastqs.txt', 'w') as f:
    for fq_path in validated_files:
        f.write(fq_path + '\n')

```

```
EOF
''''
}
```

3.6. modules/collate_samplesheet.nf

- **Purpose:** Gathers the fetched metadata and the file paths of the successfully validated FASTQ files, then organizes them into a single, standardized samplesheet (CSV format). This samplesheet is the primary output for downstream analysis.
- **Input:** path metadata_file, path validated_fastqs_list
- **Output:** path final_samplesheet_csv
- **Script:** A Python script using pandas to merge data and write the CSV.

Code snippet

```
// modules/collate_samplesheet.nf
process COLLATE_SAMPLESHEET {
  tag "Collate final samplesheet"
  input:
    path metadata_file
    path validated_fastqs_list
  output:
    path "samplesheet.csv", emit: final_samplesheet
  script:
    '''
    python <<EOF
    import json
    import pandas as pd
    import os
    import sys

    with open('$metadata_file', 'r') as f:
      metadata_raw = json.load(f)

    # Read validated FASTQ paths
```

```

fastq_paths_map = {}
with open('$validated_fastqs_list', 'r') as f:
    for line in f:
        path = line.strip()
        # Extract run_accession from the path (assuming the download module names files
correctly)
        # Example: 'SRR1234567/SRR1234567_1.fastq.gz' -> 'SRR1234567'
        run_acc = os.path.basename(os.path.dirname(path))
        if run_acc: # Ensure run_acc is not empty (e.g., if files were in current dir)
            if run_acc not in fastq_paths_map:
                fastq_paths_map[run_acc] = []
            fastq_paths_map[run_acc].append(path)
        else: # Fallback if path is just filename (e.g. SRR1234567_1.fastq.gz)
            filename_parts = os.path.basename(path).split('_')
            if len(filename_parts) > 1 and filename_parts[0].startswith(('SRR', 'ERR', 'DRR')):
                run_acc_from_filename = filename_parts[0]
                if run_acc_from_filename not in fastq_paths_map:
                    fastq_paths_map[run_acc_from_filename] = []
                fastq_paths_map[run_acc_from_filename].append(path)
            else:
                print(f"Warning: Could not infer run accession from path {path}", file=sys.stderr)

```

```

# Prepare data for samplesheet
samplesheet_data = []
for entry in metadata_raw:
    run_acc = entry.get('run_accession')
    if run_acc in fastq_paths_map:
        fastq_files = sorted(fastq_paths_map[run_acc]) # Ensure _1 then _2 for paired-end

    # Construct samplesheet entry
    sample_entry = {
        'sample_id': entry.get('sample_accession'),
        'run_id': run_acc,
        'fastq_1': fastq_files[0] if len(fastq_files) > 0 else "",
        'fastq_2': fastq_files[1] if len(fastq_files) > 1 else "",
        'study_id': entry.get('study_accession'),
        'library_strategy': entry.get('library_strategy'),
        'library_source': entry.get('library_source'),
        'library_selection': entry.get('library_selection'),
        'library_layout': entry.get('library_layout'),
    }

```

```

        'platform': entry.get('platform'),
        'instrument_model': entry.get('instrument_model'),
        'ENA_md5_checksum': entry.get('md5'), # Original ENA-provided MD5
        # Add other metadata fields as desired
    }
    samplesheet_data.append(sample_entry)
else:
    print(f"Warning: No validated FASTQ files found for run {run_acc}. Skipping this entry in
samplesheet.", file=sys.stderr)

df_samplesheet = pd.DataFrame(samplesheet_data)
df_samplesheet.to_csv('samplesheet.csv', index=False)
EOF
"""
}

```

How to Use the Workflow:

1. **Save the files:**
 - Create a directory, e.g., `ngs_fetch_workflow/`.
 - Place `main.nf` inside `ngs_fetch_workflow/`.
 - Create `ngs_fetch_workflow/conf/` and place `modules.config` inside it.
 - Create `ngs_fetch_workflow/modules/` and place all `parse_input_ids.nf`, `fetch_metadata.nf`, `download_fastq_direct.nf`, `download_fastq_sra.nf`, `validate_md5.nf`, `collate_samplesheet.nf` inside it.
2. **Prepare your input CSV file:** Create a file named, for example, `my_sample_ids.csv` in the `ngs_fetch_workflow/` directory:

Code snippet

```

ID
SRR1000001
ERR2000002
DRR3000003

```

(Replace with actual accessible database IDs for testing).

3. **Run Nextflow:** Navigate to the `ngs_fetch_workflow/` directory in your terminal and execute:

Bash

```
nextflow run main.nf \  
  --input_csv my_sample_ids.csv \  
  --outdir fetched_data \  
  -profile docker # or -profile singularity if you prefer Singularity
```

This command tells Nextflow to run main.nf, pass your CSV as input, specify an output directory, and use Docker (or Singularity) containers as defined in modules.config.

This step-by-step outline provides a comprehensive structure for your Nextflow workflow, incorporating best practices for modularity and containerization. Remember to adapt the Python scripts, especially for parsing metadata and file naming, to precisely match the structure of the ENA API responses and your desired output organization.

Response from Deepseek-v3:

Here's a step-by-step modular design for a standalone Nextflow workflow to fetch metadata and FASTQ files from public repositories:

1. Input Specification

CSV Format Example

csv

```
sample_id,project,condition  
SRR123456,PRJNA123,control  
ERR789012,PRJEB456,treatment
```

2. Workflow Modules

Module 1: CSV Parser

- Function: Extract sample IDs and optional metadata
- Tools: Built-in Nextflow `splitCsv`
- Output Channel: Emits tuples `(sample_id, project, condition)`

```
nextflow
```

```
Channel.fromPath(params.input_csv)
```

```
| splitCsv(header: true)
```

```
| map { row -> tuple(row.sample_id, row.project, row.condition) }
```

```
| set { sample_ch }
```

Module 2: Metadata Fetcher

- Methods:
 - ENA: curl to ENA API
 - SRA: pysradb or entrez-direct
 - GEO: geofetch (requires Python)
- Output: JSON files with full metadata

```
process fetch_metadata {
```

```
    input:
```

```
        tuple val(sample_id), val(project), val(condition)
```

```
    output:
```

```
        tuple val(sample_id), val(project), val(condition), path("${sample_id}.json")
```

```
    script:
```

```
        """
```

```
        # ENA API example
```

```
        curl -s "https://www.ebi.ac.uk/ena/portal/api/filereport?\
```

```
accession=${sample_id}&\
```

```
fields=run_accession,instrument_model,library_layout&\
```

```
format=json" > ${sample_id}.json
```



```
""
}
```

Module 3: FASTQ Downloader

- Strategies:
 - Direct ENA FTP: wget from ENA
 - SRA Toolkit: prefetch + fasterq-dump
 - Parallelization: Use --max-concurrent-downloads

```
process download_fastq {
```

```
  input:
```

```
    tuple val(sample_id), val(project), val(condition), path(meta)
```

```
  output:
```

```
    tuple val(sample_id), val(project), val(condition),  
    path("fastq/${sample_id}*")
```

```
  script:
```

```
    ""
```

```
    mkdir -p fastq
```

```
    # Method 1: ENA FTP (faster)
```

```
    enaDataGet -f fastq ${sample_id} -d fastq/
```

```
    # Method 2: SRA Toolkit (fallback)
```

```
    if [ ! -f "fastq/${sample_id}_1.fastq.gz" ]; then
```

```
      prefetch ${sample_id} && \
```

```
      fasterq-dump --split-files --outdir fastq/ ${sample_id} && \
```

```
    pigz fastq/${sample_id}*.fastq
```

```
fi
```

```
"""
```

```
}
```

Module 4: Output Organizer

- Tasks:
 - Rename files using metadata
 - Create standardized directory structure
 - Generate audit log

```
process organize_output {
```

```
    input:
```

```
        tuple val(sample_id), val(project), val(condition), path(fastq), path(meta)
```

```
    output:
```

```
        path("${params.outdir}")
```

```
    script:
```

```
        """
```

```
        # Create project-specific structure
```

```
        mkdir -p ${params.outdir}/${project}/{fastq,metadata}
```

```
        # Move files
```

```
        mv fastq/* ${params.outdir}/${project}/fastq/
```

```
        mv *.json ${params.outdir}/${project}/metadata/
```

```
# Create manifest

echo "sample_id,project,condition,fastq_files" > ${params.outdir}/manifest.csv

echo "${sample_id},${project},${condition},${ls
${params.outdir}/${project}/fastq/ | tr '\n' ';')}" >>
${params.outdir}/manifest.csv

"""

}
```

3. Complete Workflow Integration

```
workflow {

  // Module 1

  Channel.fromPath(params.input_csv)

    | splitCsv(header: true)

    | map { row -> tuple(row.sample_id, row.project, row.condition) }

    | set { sample_ch }


  // Module 2 → 3 → 4

  sample_ch

    | fetch_metadata

    | download_fastq

    | organize_output

}
```

4. Configuration Tips

nextflow.config

```
process {  
  
    withName:download_fastq {  
  
        cpus = 4  
  
        memory = '8 GB'  
  
        errorStrategy = 'retry'  
  
        maxRetries = 3  
  
    }  
  
}
```

```
params {  
  
    input_csv = 'samples.csv'  
  
    outdir = 'results'  
  
}
```

5. Execution & Monitoring

Run with 10 parallel downloads

```
nextflow run main.nf --input_csv samples.csv -with-report execution.html
```

Resume interrupted runs

```
nextflow run main.nf -resume
```

6. Key Enhancements

1. Repository Detection: Auto-detect source (SRA/ENA/GEO) from ID prefixes
2. Validation: Add MD5 checksum verification
3. Cloud Integration: Direct S3/GCP bucket outputs
4. Notification System: Email/Slack alerts on completion

This modular design provides flexibility to:

- Swap download tools (ENA FTP vs SRA Toolkit)
- Handle different metadata schemas
- Scale from laptops to clusters

Would you like me to elaborate on any specific module?

W3: nf-core/methylseq: This workflow is used for Methylation (Bisulfite) sequencing data.

Instruction Only Prompt:

We want to create a Nextflow workflow for DNA methylation analysis using bisulfite-converted sequencing (BS-seq) data. The workflow should begin by pre-processing raw reads provided in a CSV file containing FASTQ input information, followed by alignment to a reference genome and comprehensive quality control assessments. Could you please provide a step-by-step outline of the modules required to construct this workflow?

Gpt-4o generated response:

Certainly! Below is a **step-by-step outline of the modules** required to construct a **Nextflow workflow for DNA methylation analysis** using **bisulfite-converted sequencing (BS-seq)** data. This workflow begins with pre-processing raw reads from a CSV file and progresses through alignment, methylation extraction, and quality control reporting.



Step-by-Step Modules for BS-seq Workflow

1. Input Parsing

- **Module:** `parse_csv_input`
 - **Purpose:** Read and validate a CSV file containing sample metadata and FASTQ file paths.
 - **Output:** Emission of structured sample channels (e.g., sample ID, read1, read2).
-

2. Raw Read Quality Control

- **Module:** `fastqc_raw`
 - **Purpose:** Assess initial read quality (e.g., per-base quality, GC content, adapter content).
 - **Input:** Raw FASTQ files.
 - **Output:** Individual QC reports for each sample.
-

3. Adapter and Quality Trimming

- **Module:** `trim_reads`
 - **Purpose:** Remove sequencing adapters and low-quality bases.
 - **Input:** Raw FASTQ files.
 - **Output:** Trimmed FASTQ files ready for alignment.
-

4. Post-trimming Quality Check

- **Module:** `fastqc_trimmed`
- **Purpose:** Re-assess read quality after trimming to confirm improvement.
- **Input:** Trimmed FASTQ files.

- **Output:** Trimmed QC reports.
-

5. Reference Genome Indexing

- **Module:** `index_reference`
 - **Purpose:** Create a bisulfite-aware index of the reference genome for alignment.
 - **Input:** Reference genome FASTA file.
 - **Output:** Indexed genome files.
-

6. Read Alignment

- **Module:** `align_bs_reads`
 - **Purpose:** Align bisulfite-treated reads to the reference genome using a bisulfite-aware aligner.
 - **Input:** Trimmed reads and indexed reference.
 - **Output:** Aligned BAM files with bisulfite-aware mappings.
-

7. Deduplication

- **Module:** `remove_duplicates`
 - **Purpose:** Identify and remove PCR duplicates from aligned BAM files to improve accuracy.
 - **Input:** BAM files.
 - **Output:** Deduplicated BAM files.
-

8. Methylation Extraction

- **Module:** `extract_methylation`
 - **Purpose:** Extract cytosine methylation information (e.g., CpG, CHG, CHH contexts) from BAM files.
 - **Input:** Deduplicated BAM files.
 - **Output:** Methylation reports, coverage files.
-

9. Additional Quality Metrics (Optional)

- **Modules:** `library_complexity`, `coverage_stats`, `alignment_metrics`
 - **Purpose:** Compute optional QC metrics such as:
 - Library complexity
 - Genome coverage
 - Alignment quality
-

10. Aggregated Reporting

- **Module:** `aggregate_qc`
 - **Purpose:** Compile all QC metrics, trimming stats, alignment reports, and methylation summaries into a single report.
 - **Output:** HTML summary report, plots, and tables.
-

11. Pipeline Audit and Provenance

- **Module:** `generate_report`

- **Purpose:** Create execution logs, DAG visualization, resource trace, and version-controlled metadata for reproducibility.
-

Output Summary

- Trimmed reads (FASTQ)
 - Aligned and deduplicated BAM files
 - Cytosine methylation reports
 - QC metrics and summary reports (HTML/PDF/TSV)
-

Would you like a scaffold of the `main.nf` workflow using these modules in Nextflow DSL2?

Role-based Prompt:

You are a bioinformatics workflow developer with expertise in building reproducible pipelines using Nextflow and nf-core modules. Your task is to design a Nextflow workflow for DNA methylation analysis using bisulfite-converted sequencing (BS-seq) data. The workflow should begin by pre-processing raw reads specified in a CSV file (containing sample IDs and FASTQ paths), then proceed to align reads to a reference genome, extract methylation information, and generate comprehensive quality control reports. Please provide a step-by-step outline of the modules required to build this workflow, using only officially available nf-core or Nextflow modules, and include their exact module names where appropriate.

Gpt-4o response:

Certainly! Below is a step-by-step outline of the modules required to build a reproducible **Nextflow workflow for DNA methylation analysis** using **bisulfite-converted sequencing (BS-seq)** data. The workflow leverages **official nf-core modules** to ensure compatibility, portability, and maintainability.

Step-by-Step Modules for BS-seq Workflow

1. Input Parsing

- **Custom Process** (no official module)
 - **Task:** Read and validate a CSV file containing sample metadata and FASTQ file paths.
 - **Implementation Suggestion:** Use a Groovy script or embedded Python/R to parse the CSV and create input channels.
-

2. Raw Read Quality Control

- **Module:** [fastqc](#)
 - **Task:** Perform initial quality control on raw FASTQ files.
-

3. Adapter and Quality Trimming

- **Module:** [trim_galore](#)
 - **Task:** Trim adapters and low-quality bases. Suitable for bisulfite sequencing protocols including PBAT and RRBS with appropriate flags.
-

4. Post-trimming Quality Control

- **Module:** [[fastqc](#)](reuse from Step 2)
 - **Task:** Assess read quality after trimming to verify improvements.
-

5. Reference Genome Indexing

- **Module:** [bismark/genomeprepare](#)
- **Task:** Generate a bisulfite-aware index of the reference genome.

6. Read Alignment

- **Module:** [bismark/align](#)
- **Task:** Align bisulfite-treated reads (single- or paired-end) to the reference genome.

7. Deduplication

- **Module:** [bismark/deduplicate](#)
- **Task:** Remove PCR duplicates from aligned BAM files to improve methylation call accuracy.

8. Methylation Extraction

- **Module:** [bismark/methylationextractor](#)
- **Task:** Extract methylation calls (CpG, CHG, CHH contexts) from deduplicated BAM files.

9. Optional QC Modules

- **Module:** [qualimap/bamqc](#)
- **Module:** [preseq/lc_extrap](#)
- **Module:** [picard/collecthsmetrics](#)
- **Task:** Compute additional quality metrics such as coverage, library complexity, and hybrid selection efficiency (if applicable).

10. Report Aggregation

- **Module:** [multiqc](#)
 - **Task:** Compile and summarize all QC metrics, alignment stats, and methylation summaries into a single HTML report.
-

Output Summary

- Quality-checked and trimmed FASTQ files
 - Aligned and deduplicated BAM files
 - Cytosine methylation context reports
 - Aggregated quality control reports (HTML, TSV)
 - Optional: Coverage stats, complexity metrics, hybrid selection metrics
-

This outline ensures the workflow is fully compatible with the nf-core module library and can be readily deployed on local, HPC, or cloud systems using Nextflow. Let me know if you'd like a DSL2 `main.nf` scaffold based on this structure.

Response from Gemini 2.5 Flash:

To create a Nextflow workflow for DNA methylation analysis using bisulfite-converted sequencing (BS-seq) data, starting with raw reads from a CSV file, you'll need to define a series of interconnected modules (processes) that handle each step from pre-processing to quality control.

Here's a step-by-step outline of the modules required, including common tools used in BS-seq analysis:

Workflow Input and Configuration

1. Input Parsing Module (READ_SAMPLES_CSV):

- **Purpose:** Reads the CSV file containing FASTQ file paths and sample metadata (e.g., sample ID, R1 FASTQ, R2 FASTQ if paired-end, and any other relevant columns like a group identifier).
- **Tools/Nextflow Features:** Nextflow's Channel.fromPath and readLines or readCsv to parse the input CSV and create a channel of sample information (e.g., a tuple (sample_id, R1_fastq, R2_fastq)). This channel will drive the rest of the workflow.

Pre-processing Modules

2. Raw Read Quality Control (FASTQC):

- **Purpose:** Assesses the quality of raw FASTQ reads.
- **Tool:** FastQC
- **Input:** Raw FASTQ files (from READ_SAMPLES_CSV output).
- **Output:** FastQC HTML reports and data directories for each sample.

3. Adapter and Quality Trimming (TRIM_GALORE):

- **Purpose:** Removes adapter sequences and low-quality bases from the reads, which is crucial for bisulfite-converted data. Trim Galore! is a wrapper around Cutadapt.
- **Tool:** Trim Galore! (internally uses Cutadapt)
- **Input:** Raw FASTQ files.
- **Output:** Trimmed FASTQ files and trimming reports.

Reference Genome Preparation

4. Reference Genome Indexing (BISMARK_INDEX / BWAMETH_INDEX):

- **Purpose:** Creates a bisulfite-aware index of the reference genome, essential for alignment of bisulfite-converted reads. This is typically a one-time step.
- **Tools:** Bismark_genome_preparation (for Bismark) or bwa-meth index (for bwa-meth).
- **Input:** Reference genome FASTA file.
- **Output:** Indexed reference genome directory.

Alignment and Methylation Calling Modules

5. **Read Alignment (BISMARK_ALIGN / BWAMETH_ALIGN):**
 - **Purpose:** Aligns the bisulfite-converted reads to the indexed reference genome.
 - **Tools:**
 - Bismark (often using Bowtie2 or HISAT-3n as its aligner).
 - bwa-meth.
 - **Input:** Trimmed FASTQ files, Reference genome index.
 - **Output:** Aligned BAM/SAM files, alignment reports (including mapping statistics and bisulfite conversion rates).
6. **PCR Duplicate Removal (DEDUPLICATE_BAM):**
 - **Purpose:** Identifies and removes PCR duplicates from the aligned BAM files to prevent overcounting.
 - **Tools:** deduplicate_bismark (if using Bismark), or general tools like samblaster or Picard MarkDuplicates.
 - **Input:** Aligned BAM/SAM files.
 - **Output:** Deduplicated BAM files.
7. **Methylation Extraction (BISMARK_METHYL_EXTRACTOR / METHYLDACKEL_EXTRACT):**
 - **Purpose:** Extracts methylation calls (e.g., CpG, CHG, CHH context counts) from the aligned and deduplicated reads.
 - **Tools:**
 - bismark_methylation_extractor (for Bismark).
 - MethylDackel extract (for bwa-meth).
 - **Input:** Deduplicated BAM files.
 - **Output:** Methylation call files (e.g., BedGraph, cytosine report), methylation bias reports (M-bias plots).

Quality Control and Reporting Modules

8. **Post-Alignment Quality Control (QUALIMAP_BAMQC - Optional, but good practice):**
 - **Purpose:** Provides comprehensive quality metrics for the aligned BAM files. While Bismark/bwa-meth provide some stats, QualiMap offers a deeper dive into alignment quality.
 - **Tool:** QualiMap
 - **Input:** Deduplicated BAM files.
 - **Output:** QualiMap reports.
9. **Methylation Bias Plotting (METHYLDACKEL_MBIAS - if not already generated):**
 - **Purpose:** Generates plots illustrating any methylation bias across the read length.

- **Tool:** MethylDackel mbias
 - **Input:** Deduplicated BAM files.
 - **Output:** Methylation bias plots (e.g., SVG).
10. **Aggregate Quality Control Reports (MULTIQC):**
- **Purpose:** Gathers and consolidates all the quality control metrics and reports from FastQC, Trim Galore!, Bismark/bwa-meth, QualiMap, etc., into a single, interactive HTML report.
 - **Tool:** MultiQC
 - **Input:** All relevant log files and reports generated by the preceding QC and analysis modules (e.g., fastqc_data.zip, trim_galore_report.txt, bismark_report.txt, methylDackel_report.txt, qualimap_report.txt).
 - **Output:** A single MultiQC HTML report.

Workflow Structure Considerations

- **Channels:** Each module will typically consume one or more input channels and emit one or more output channels. The output of one module becomes the input for the next.
- **Parameters:** Allow users to specify the input CSV file, reference genome, choice of aligner (Bismark/bwa-meth), output directory, and other tool-specific options.
- **Containers:** Utilize Docker or Singularity containers for each tool to ensure reproducibility and simplify dependency management.
- **Error Handling:** Implement robust error handling (e.g., errorStrategy) to manage failed processes.
- **Resource Management:** Define resource requirements (CPU, memory, time) for each process to optimize execution on different computing environments.

This outline provides a solid foundation for constructing your Nextflow BS-seq analysis workflow. Many of these modules are available directly as nf-core modules, which are well-tested and follow best practices.

Response from DeepSeek-V3:

Certainly! Below is a step-by-step outline for building a Nextflow workflow for BS-seq (bisulfite sequencing) data analysis, including preprocessing, alignment, methylation calling, and quality control.

Nextflow Workflow for BS-seq Analysis

1. Input Handling

- Input: A CSV file containing sample metadata (e.g., `sample_name`, `fastq1`, `fastq2` for paired-end reads).
- Module: Parse CSV and validate inputs.

Example CSV Format

```
csv
sample,fastq_1,fastq_2
sample1,/path/to/sample1_R1.fastq.gz,/path/to/sample1_R2.fastq.gz
sample2,/path/to/sample2_R1.fastq.gz,/path/to/sample2_R2.fastq.gz
```

2. Preprocessing (Quality Control & Trimming)

- Tool: `FastQC` (raw read QC) + `Trim Galore!` (adapter trimming & quality filtering).
- Purpose:
 - Check sequencing quality.
 - Remove adapters and low-quality bases.
 - Retain reads post-trimming for alignment.

Module Steps

1. Run `FastQC` on raw reads.
2. Run `Trim Galore!` (with `--paired` for PE data, `--rrbs` for RRBS).
3. Run `FastQC` again on trimmed reads.

3. Alignment to Reference Genome

- Tools: `Bismark` (Bowtie2/Hisat2) or `bwa-meth`.
- Purpose: Align bisulfite-converted reads to a reference genome.

Module Steps

1. Bismark Alignment:


```
Bash
```

```
bismark --genome <ref_dir> -1 trimmed_R1.fq -2 trimmed_R2.fq
```

2. Deduplication (optional, for WGBS):

```
Bash
```

```
deduplicate_bismark --bam sample1_trimmed_bismark_bt2.bam
```

3. Methylation Extraction:

```
bash
```

```
bismark_methylation_extractor --bedGraph sample1_trimmed_bismark_bt2.bam
```

4. Methylation Calling & Coverage Metrics

- Tool: `Bismark` or `MethylDackel` (for coverage stats).
- Purpose: Generate per-CpG methylation calls and coverage reports.

Module Steps

1. Generate methylation bedGraph files.
2. Compute genome-wide coverage statistics.
3. (Optional) Aggregate results into a methylation matrix (for downstream analysis).

5. Quality Control & Reporting

- Tools: `MultiQC`, `Qualimap`, custom R/Python scripts.
- Purpose: Summarize all QC metrics.

Module Steps

1. MultiQC: Aggregate reports from `FastQC`, `Trim Galore!`, `Bismark`.
2. Bismark Summary Report:

```
bash
```

```
bismark2report
```

3. Methylation Bias Plot (check for 5'/3' bias).

4. PCA/Clustering (if multiple samples).

6. (Optional) Differential Methylation Analysis

- Tool: `MethylKit` (R) or `DSS`.
- Purpose: Compare methylation between groups (e.g., case vs. control).

Module Steps

1. Merge methylation calls across samples.
2. Perform DMR (Differentially Methylated Regions) analysis.
3. Generate heatmaps/volcano plots.

Suggested Nextflow Implementation

1. Directory Structure

workflow/

```
|— main.nf      # Main workflow script
|— modules/     # Custom modules
| |— fastqc.nf  # FastQC process
| |— trim_galore.nf # Trimming process
| |— bismark.nf  # Alignment & methylation
| |— multiqc.nf  # QC reporting
|— conf/        # Configuration files
|— data/        # Input CSV & FASTQs
```

2. Example `main.nf` Skeleton

```
params.csv = "samples.csv" // Input CSV
```

```
params.genome = "/path/to/genome/"
```

```

include { FASTQC } from './modules/fastqc'

include { TRIM_GALORE } from './modules/trim_galore'

include { BISMARK } from './modules/bismark'

include { MULTIQC } from './modules/multiqc'


workflow {

    // Read CSV input

    Channel.fromPath(params.csv)

        | splitCsv(header: true)

        | map { row -> tuple(row.sample, row.fastq_1, row.fastq_2) }

        | set { samples_ch }


    // Run pipeline

    FASTQC(samples_ch)

    TRIM_GALORE(FASTQC.out)

    BISMARK(TRIM_GALORE.out, params.genome)

    MULTIQC(BISMARK.out)

}

```

Final Notes

- Reference Genome: Must be pre-indexed for Bismark (bismark_genome_preparation).
- Containers: Use Docker/Singularity for reproducibility.
- Parallelization: Nextflow automatically parallelizes per sample.

Would you like help with any specific module (e.g., Bismark config or CSV parsing)?

W4: nf-core/rnaseq: This workflow is used to analyse RNA sequencing data with a reference genome and annotation.

Instruction-Only Prompt:

We want to create a Nextflow workflow for RNA-seq analysis that performs quality control, adapter trimming, alignment, quantification, and differential expression analysis. The workflow should accept a CSV sample sheet containing FASTQ files link and genome references as input. It must include quality checks, reference genome indexing if needed, and gene/transcript quantification and produce a gene expression matrix and extensive QC report using appropriate tools. Please provide a step-by-step outline of the modules to construct this workflow.

Gpt-4o Generated Response:

Certainly. Below is a **step-by-step outline** to construct a **Nextflow workflow for RNA-seq analysis**, meeting all the specified requirements:

Nextflow Workflow for RNA-seq Analysis

1. Input Parsing

- **Input:** CSV sample sheet with columns:
 - `sample`, `fastq_1`, `fastq_2`, `genome_fasta`, `annotation_gtf`
- **Process:** Parse the CSV to extract download paths or local file references for FASTQ files and genome resources.

2. FASTQ Quality Control

- **Tool:** `FastQC`
- **Purpose:** Assess the raw sequencing quality of input reads.
- **Output:** HTML and `.zip` quality reports for each FASTQ file.

3. Adapter Trimming

- **Tool:** Cutadapt or Trim Galore
- **Purpose:** Remove adapter sequences and trim low-quality bases.
- **Output:** Cleaned FASTQ files.

4. Post-trimming Quality Control

- **Tool:** FastQC
- **Purpose:** Re-evaluate quality of trimmed reads.
- **Output:** New QC reports for trimmed FASTQ files.

5. Reference Genome Indexing

- **Tool:** STAR or HISAT2 index builder
- **Purpose:** Index the reference genome if index files are not provided.
- **Trigger:** Conditional step based on presence of index.
- **Output:** Genome index directory.

6. Read Alignment

- **Tool:** STAR or HISAT2
- **Purpose:** Align cleaned reads to the reference genome.

- **Output:** Aligned BAM files.
-

7. BAM Sorting and Indexing

- **Tool:** `Samtools`
 - **Purpose:** Sort and index aligned reads for downstream quantification.
 - **Output:** Sorted and indexed BAM files.
-

8. Gene/Transcript Quantification

- **Tool:** `featureCounts` (gene-level) or `Salmon/Kallisto` (transcript-level)
 - **Purpose:** Quantify gene or transcript abundance from alignments.
 - **Input:** Sorted BAMs or cleaned FASTQs + annotation (GTF).
 - **Output:** Raw count matrix per sample.
-

9. Differential Expression Analysis

- **Tool:** `DESeq2` or `edgeR` (via R script)
 - **Purpose:** Identify differentially expressed genes between sample groups.
 - **Input:** Count matrix + sample metadata (e.g., condition/group info).
 - **Output:** Tables of log-fold changes, p-values, normalized counts, visualizations.
-

10. Aggregated Quality Control Report

- **Tool:** MultiQC
 - **Purpose:** Generate a unified summary report of all QC steps.
 - **Input:** Logs and results from FastQC, trimming, alignment, and quantification.
 - **Output:** Single HTML report for easy review.
-

11. Output Delivery

- Final outputs include:
 - Gene/transcript count matrix
 - Differential expression tables and plots
 - Cleaned FASTQ and BAM files
 - QC reports (FastQC, MultiQC)
 - Log files and execution trace
-

Let me know if you'd like this as a Nextflow `main.nf` template or a `workflow.nf` + `config` structure.

Gemini 2.5 Flash generated response:

Creating a Nextflow workflow for RNA-seq analysis involves several key steps, each typically handled by a dedicated module. This modular approach ensures flexibility, reproducibility, and scalability.

Workflow Input and Configuration

1. **Input Parsing Module (READ_SAMPLESHEET_CSV):**
 - **Purpose:** Reads the CSV sample sheet, which contains information about each

sample, including sample ID, FASTQ file paths (R1 and R2 for paired-end), and potentially group information for differential expression. It also handles the genome references.

- **Tools/Nextflow Features:** Nextflow's Channel.fromPath and readCsv to parse the CSV. This process will emit a channel of tuples, with each tuple representing a sample's metadata and FASTQ files, driving the subsequent processes. The genome reference path can be passed as a parameter.
-

Pre-processing Modules

2. Raw Read Quality Control (FASTQC):

- **Purpose:** Assesses the quality of raw FASTQ reads before any modifications. This helps identify potential issues like adapter contamination, low-quality bases, or sequencing biases.
- **Tool: FastQC** 
- **Input:** Raw FASTQ files for each sample (from the input parsing module).
- **Output:** FastQC HTML reports and data directories.

3. Adapter and Quality Trimming (TRIMGALORE / FASTP):

- **Purpose:** Removes adapter sequences and low-quality bases from the ends of the reads. This step is crucial for improving alignment accuracy and downstream analysis.
 - **Tools: Trim Galore!** (a wrapper for Cutadapt) or **Fastp** are common choices.
 - **Input:** Raw FASTQ files.
 - **Output:** Trimmed FASTQ files and trimming reports.
-

Reference Preparation and Alignment Modules

4. Reference Genome Indexing (STAR_INDEX / SALMON_INDEX / KALLISTO_INDEX):

- **Purpose:** Prepares the reference genome and its associated annotation (e.g., GTF/GFF file) for efficient alignment or pseudo-alignment. This step is typically performed once per reference genome.
- **Tools:**
 - **STAR** for splice-aware alignment.
 - **Salmon** or **Kallisto** for pseudo-alignment/quantification.
- **Input:** Reference genome FASTA file and gene annotation (GTF/GFF) file.

- **Output:** Indexed genome directories.
 - 5. **Read Alignment / Pseudo-alignment (STAR_ALIGN / SALMON_QUANT / KALLISTO_QUANT):**
 - **Purpose:** Maps the trimmed reads to the reference genome or quantifies transcript abundance directly.
 - **Tools:**
 - **STAR** for alignment, producing BAM files. This is a common choice for comprehensive alignment.
 - **Salmon** or **Kallisto** for faster, alignment-free quantification (pseudo-alignment), directly producing transcript abundance estimates.
 - **Input:** Trimmed FASTQ files, indexed genome.
 - **Output:** Aligned BAM/SAM files (for STAR) or quantification files (e.g., quant.sf for Salmon/Kallisto), along with alignment/quantification logs.
-

Quantification and Differential Expression Modules

- 6. **Gene/Transcript Quantification (if not done in alignment step) (FEATURECOUNTS / HTSEQ_COUNT):**
 - **Purpose:** Counts reads that map to specific genes or transcripts to generate raw gene/transcript counts. This step is needed if a full aligner like STAR was used.
 - **Tools:** **featureCounts** (part of Subread) or **HTSeq-count**.
 - **Input:** Aligned BAM files, gene annotation (GTF/GFF) file.
 - **Output:** Raw count matrix for genes/transcripts.
 - 7. **Differential Expression Analysis (DESEQ2 / EDGER / LIMMA_VOOM):**
 - **Purpose:** Identifies genes or transcripts that are significantly up- or down-regulated between different experimental conditions (defined in the sample sheet).
 - **Tools:** **DESeq2**, **edgeR**, or **limma-voom** (R packages). These would typically be run within a Nextflow process executing an R script. 
 - **Input:** Raw count matrix, sample metadata (from input parsing).
 - **Output:** Tables of differentially expressed genes (log2 fold change, p-value, adjusted p-value), normalized expression values.
-

Quality Control Reporting Module

- 8. **Aggregate Quality Control Reports (MULTIQC):**

- **Purpose:** Consolidates and summarizes all the quality control metrics and reports generated by the various upstream tools (FastQC, Trim Galore!, STAR, Salmon/Kallisto logs) into a single, interactive HTML report. This provides a comprehensive overview of the experiment's quality. 
 - **Tool: MultiQC**
 - **Input:** All relevant log files and reports from previous QC and analysis modules.
 - **Output:** A single MultiQC HTML report.
-

Nextflow Workflow Considerations:

- **Modular Design:** Each step above corresponds to a Nextflow process or a group of processes in a subworkflow.
- **Channels:** Data flow between modules using Nextflow channels, ensuring parallelism and efficient data handling.
- **Parameters:** The workflow should be parameterized to allow users to specify input paths (CSV sample sheet, reference genome FASTA, GTF), tool-specific options, and output directories.
- **Containers:** Utilize **Docker** or **Singularity** containers for each tool to ensure reproducibility across different computing environments and simplify dependency management.
- **Resource Management:** Define appropriate CPU, memory, and time requirements for each process.

DeepSeek-V3 Generated Response:

Below is a step-by-step outline for building a Nextflow RNA-seq workflow that performs QC, trimming, alignment, quantification, and differential expression analysis using a CSV sample sheet as input.

1. Input Requirements

- CSV Sample Sheet (Example format):

```
sample,fastq_1,fastq_2,strandedness,genome
sample1,/path/sample1_R1.fastq.gz,/path/sample1_R2.fastq.gz,reverse,G
RCh38
sample2,/path/sample2_R1.fastq.gz,/path/sample2_R2.fastq.gz,reverse,G
RCh38
```

- Reference Genome (Auto-download if not indexed)
- GTF/GFF Annotation (Required for quantification)

2. Step-by-Step Module Outline

A. Initial Setup & Input Parsing

1. Parse CSV Sample Sheet
 - Use Nextflow to read the CSV and validate paths.
 - Check for paired/single-end reads and strandedness.
2. Download & Index Reference Genome (if needed)
 - Tools: `curl/wget`, `STAR/HISAT2`, `samtools`
 - Output: Genome index files (STAR, HISAT2, Salmon decoy-aware index).

B. Quality Control (QC)

3. Raw Read QC (`FastQC`)
 - Run FastQC on raw FASTQ files.
 - Output: HTML/PDF reports per sample.
4. Adapter Trimming (`Trim Galore!` / `Cutadapt`)
 - Remove adapters and low-quality bases.
 - Output: Trimmed FASTQ files.
5. Post-Trimming QC (`FastQC` again)
 - Ensure trimming improved data quality.

C. Alignment

6. Alignment to Reference Genome
 - Option 1: `STAR` (splice-aware, best for most RNA-seq)
 - Option 2: `HISAT2` (faster, good for small genomes)
 - Output: BAM files (sorted & indexed).
7. Alignment QC (`Qualimap`, `RSeQC`, `dupRadar`)
 - Check mapping rates, duplicates, and strandedness.

D. Quantification

8. Gene/Transcript Quantification

- Gene-level counts: `featureCounts` (from BAM)
- Transcript-level counts: `Salmon` (alignment-free)
- Output: Count matrix (genes/transcripts × samples).

E. Differential Expression (Optional)

9. Differential Expression (`DESeq2`, `edgeR`, `limma-voom`)
 - Input: Count matrix from `featureCounts`/`Salmon`.
 - Output: DE results (CSV/HTML reports).

F. Final Reporting

10. Aggregate QC Reports (`MultiQC`)
 - Combine all QC metrics (FastQC, Qualimap, Salmon, etc.).
 - Output: Single HTML report.

3. Nextflow Implementation Outline

`main.nf` (Workflow Structure)

```
params {
  samplesheet = "samples.csv"
  genome = "GRCh38" // Default, can be overridden per sample
  gtf = "gencode.v44.annotation.gtf"
}

// Parse samplesheet
Channel.fromPath(params.samplesheet)
  | splitCsv(header: true)
  | map { row -> [ row.sample, [ row.fastq_1, row.fastq_2 ], row.strandedness, row.genome
  ] }
  | set { samples_ch }

// Reference genome indexing (if needed)
process IndexGenome {
  input: val genome
```

```

    output: path("genome_index")
    script: "STAR --runMode genomeGenerate ..."
}

// Main workflow
workflow {
    // QC & Trimming
    FastQC(samples_ch)
    TrimGalore(samples_ch)

    // Alignment
    STAR(TrimGalore.out, IndexGenome.out)

    // Quantification
    featureCounts(STAR.out, params.gtf)

    // Differential Expression (if specified)
    if (params.run_deseq2) {
        DESeq2(featureCounts.out)
    }

    // MultiQC
    MultiQC(FastQC.out, TrimGalore.out, STAR.out, featureCounts.out)
}

```

modules.config (Tool Versions & Containers)

```

process {
    withName: 'FastQC' { container = 'quay.io/biocontainers/fastqc:0.12.1' }
    withName: 'STAR' { container = 'quay.io/biocontainers/star:2.7.10a' }
    // ... other tool configs
}

```

nextflow.config (Execution Profiles)

```

profiles {
    local { process.executor = 'local' }
}

```

```
slurm { process.executor = 'slurm' }  
docker { docker.enabled = true }  
singularity { singularity.enabled = true }  
}
```

4. Expected Outputs

Step	Output Files
QC	<code>fastqc/</code> , <code>multiqc_report.html</code>
Trimmed Reads	<code>trimmed/*_val_{1,2}.fq.gz</code>
Alignment	<code>star_aligned/*.bam</code> , <code>star_logs/*Log.final.out</code>
Quantification	<code>featureCounts/counts.txt</code> , <code>salmon/quant.sf</code>
DE Analysis	<code>deseq2/results.csv</code> , <code>volcano_plots.pdf</code>

5. Enhancements (Optional)

- Custom Genome Support (Allow user-defined genomes).
- Batch Correction (`ComBat`, `sva`).
- Alternative Splicing (`rMATS`, `SUPPA2`).
- Single-Cell RNA-seq Mode (If needed).

Would you like a detailed implementation for any specific step?

W5: nf-core/phyloplace: a bioinformatics best-practice analysis pipeline that performs phylogenetic placement with EPA-NG.

Instruction Only Prompt: We want to create a Nextflow workflow for the phylogenetic placement of query sequences onto a reference tree, supporting two operational modes. In **placement-only mode**, the workflow uses a query FASTA file, a reference alignment, a rooted phylogenetic tree in Newick format, an evolutionary model, and optionally a taxonomy file for classification. In **search-plus-placement mode**, it begins with a large unfiltered FASTA file and HMM profiles to identify candidate sequences, followed by alignment, placement, classification, and reporting using the same reference inputs. The workflow aligns selected sequences, places them onto the tree, classifies their positions, and generates summary reports and visualizations. Could you please provide a step-by-step outline of the modules required to construct this workflow?

GPT-4o Response:

Certainly! Below is a **step-by-step outline** of the **modular components** required to build a **Nextflow workflow** for phylogenetic placement of query sequences onto a reference tree, supporting both **placement-only** and **search-plus-placement** modes.

Step-by-Step Modules for Phylogenetic Placement Workflow

1. Input Handling and Mode Detection

- **Objective:** Parse inputs and determine the operational mode.
- **Inputs:**
 - Placement-only mode: query FASTA, reference alignment, rooted tree (Newick), model file, optional taxonomy
 - Search-plus-placement mode: unfiltered FASTA, HMM profiles, reference alignment, rooted tree, model, optional taxonomy

- **Tasks:**
 - Validate all input files
 - Infer mode based on presence/absence of unfiltered FASTA and HMM profiles
 - Prepare metadata for downstream steps
-

2. (Optional) HMM-based Candidate Selection

(Only in search-plus-placement mode)

- **Objective:** Identify candidate sequences from the large input using HMM profiles.
 - **Tool:** `hmmsearch`
 - **Tasks:**
 - Search unfiltered FASTA with each HMM profile
 - Select top-scoring hits per profile
 - Extract and compile candidate sequences into a new FASTA for alignment
-

3. Sequence Alignment to Reference

- **Objective:** Align candidate or direct query sequences to the reference alignment.
- **Tools** (configurable):
 - `mafft --add`, `clustalo`, or `hmmalign`
- **Tasks:**
 - Integrate new sequences into the reference MSA

- Ensure alignment consistency with phylogenetic requirements (e.g., format, gap handling)
 - Optionally mask poorly aligned regions
-

4. Reference Tree and Model Preparation

- **Objective:** Validate and preprocess reference components for placement.
 - **Tasks:**
 - Ensure reference tree is properly rooted and in Newick format
 - Confirm model file compatibility (e.g., RAxML or IQ-TREE format)
 - Validate consistency between tree tips and alignment headers
-

5. Phylogenetic Placement

- **Objective:** Place aligned query sequences onto the reference tree.
 - **Tool:** `epa-ng` or another likelihood-based placement tool
 - **Inputs:** aligned query + reference MSA, rooted tree, evolutionary model
 - **Outputs:** placement file (e.g., `.jplace` format)
-

6. Tree Grafting

- **Objective:** Integrate placed sequences into the reference tree structure.
- **Tool:** tree-grafting utility (e.g., based on `.jplace` format)
- **Tasks:**

- Graft placed queries onto the reference tree
 - Generate an updated Newick tree with both reference and query tips
-

7. (Optional) Taxonomic Classification

- **Objective:** Assign taxonomic labels to query sequences based on placement.
 - **Inputs:** taxonomy mapping file (e.g., reference tip → taxonomy table)
 - **Tasks:**
 - Map placement likelihoods to taxonomic ranks
 - Produce a classification summary for all placed queries
-

8. Reporting and Visualization

- **Objective:** Summarize results and provide output suitable for downstream interpretation.
- **Tools:**
 - **MuLtiQC** for pipeline-wide statistics
 - Tree visualization utilities or scripts
- **Outputs:**
 - Summary statistics (alignment, placement, classification)
 - Final tree (Newick)
 - Placement files
 - Classification table (if applicable)
 - Interactive or publication-ready reports

9. Output Organization and Execution Metadata

- **Objective:** Ensure outputs are clean, reproducible, and well-documented.
- **Tasks:**
 - Organize outputs into structured directories (e.g., `alignment/`, `placement/`, `classification/`, `reports/`)
 - Generate:
 - DAG and execution timeline
 - Software version report
 - Parameter configuration used

Example Directory Structure

```
workflow/
├── main.nf
├── modules/
│   ├── input_parsing/
│   ├── hmm_search/
│   ├── alignment/
│   ├── placement/
│   ├── tree_grafting/
│   ├── classification/
│   └── reporting/
├── conf/
│   └── params.config
├── bin/
│   └── custom_scripts.sh
├── assets/
│   └── test_data/
```

Let me know if you'd like help generating the `main.nf` scaffold, configuration files, or implementing any of the modules listed.

Gemini 2.5 Flash Generated Response:

To construct this Nextflow workflow for phylogenetic placement, you'll need the following modules, organized by their function and operational mode:

1. Core Modules (Used in Both Modes)

These modules are fundamental to both the "placement-only" and "search-plus-placement" modes.

- **FASTA_ALIGN:** This module will take a query FASTA file and a reference alignment, then align the query sequences to the reference.
 - **Input:** Query FASTA, Reference Alignment.
 - **Tool:** Likely **MAFFT** for multiple sequence alignment, or **HMMER**'s `hmmalign` if aligning to a profile.
 - **Output:** Aligned query sequences.
 - **PHYLOGENETIC_PLACEMENT:** This module performs the actual placement of the aligned query sequences onto the reference tree.
 - **Input:** Aligned Query Sequences, Reference Tree (Newick), Evolutionary Model.
 - **Tool:** **EPA-NG** (Evolutionary Placement Algorithm - Next Generation).
 - **Output:** Placement file (e.g., JPLACE format).
 - **PLACEMENT_SUMMARY:** This module processes the placement results to generate a summarized output, including a grafted tree and classification.
 - **Input:** Placement File, Reference Tree, (Optional) Taxonomy File.
 - **Tool:** **Gappa** (Generalized ANcestral Placement and Phylogeny Analysis).
 - **Output:** Grafted tree (Newick), Classification file, Heat tree data.
 - **REPORT_GENERATION:** This module creates various reports and visualizations of the placement results.
 - **Input:** Grafted Tree, Classification File, Heat Tree Data.
 - **Tool:** Potentially custom scripts or other visualization tools like **R** or **Python** libraries.
 - **Output:** HTML reports, graphical visualizations (e.g., phylogenetic trees with placements highlighted).
-

2. Search-Plus-Placement Specific Modules

These modules are only activated when the workflow is running in "search-plus-placement" mode.

- **HMM_SEARCH:** This initial module screens a large, unfiltered FASTA file to identify candidate sequences using HMM profiles.
 - **Input:** Unfiltered FASTA file, HMM Profiles.
 - **Tool:** HMMER (hmmsearch or phmmer).
 - **Output:** FASTA file containing identified candidate sequences (filtered query sequences).
-

3. Workflow Logic and Branching

The Nextflow workflow script will need to incorporate conditional logic to handle the two operational modes:

- **Conditional Execution:** An **if statement** or similar mechanism based on a workflow parameter (e.g., `--mode placement-only` or `--mode search-plus-placement`) will determine which set of modules to execute.
- **Input Handling:** Define specific **parameters** for each required input file (e.g., `--query_fasta`, `--reference_alignment`, `--reference_tree`, `--hmm_profiles`, `--taxonomy`).
- **Channel Management:** Ensure that the outputs of one module are correctly passed as inputs to the next, forming a seamless pipeline. For example, the output of HMM_SEARCH will feed into FASTA_ALIGN in "search-plus-placement" mode, while in "placement-only" mode, FASTA_ALIGN will take the initial `--query_fasta`.

By structuring the workflow with these distinct modules and clear conditional logic, you can effectively manage the different operational modes and ensure a robust phylogenetic placement pipeline. 🌳📊

Response from DeepSeek-V3:

Below is a step-by-step outline of the Nextflow workflow for phylogenetic placement, supporting both placement-only and search-plus-placement modes. The workflow is modular, allowing flexibility in tools and configurations.

Nextflow Workflow for Phylogenetic Placement

1. Input Parameters & Validation

- Inputs:
 - `--mode` (placement or search_placement)
 - `--queries` (FASTA/FASTQ for placement mode)
 - `--unfiltered_fasta` (Large FASTA for search mode)
 - `--reference_alignment` (MSA in FASTA)
 - `--reference_tree` (Rooted Newick)
 - `--model` (Evolutionary model, e.g., GTR+G)
 - `--taxonomy` (Optional TSV for classification)
 - `--hmm_profiles` (HMMs for search mode)
 - Validation:
 - Check file formats (FASTA, Newick, HMM validity).
 - Verify tree is rooted.
-

2. Modules for Placement-Only Mode

A. Sequence Alignment (if needed)

- Tool: MAFFT / HMMER
- Input: Query sequences (`--queries`).
- Output: Aligned FASTA.

B. Phylogenetic Placement

- Tools:
 - EPA-ng (for maximum-likelihood placement)
 - pplacer (alternative)
- Input:
 - Aligned queries + `--reference_alignment` + `--reference_tree` + `--model`.
- Output:

- `.jplace` (placement file)
- Updated tree with placements (`newick`).

C. Taxonomic Classification (Optional)

- Tool: `gappa / taxit`
- Input: `.jplace + --taxonomy`.
- Output:
 - Taxonomic assignments (`.tsv`).
 - Annotated tree (e.g., `iTOL`-compatible).

D. Reporting & Visualization

- Tools:
 - `ETE3` (tree visualization)
 - `R/Python` (diversity stats, reports).
 - Output:
 - HTML/PDF summaries.
 - Interactive tree views.
-

3. Modules for Search-Plus-Placement Mode

A. Sequence Search (HMM Filtering)

- Tool: `hmmsearch` (HMMER)
- Input:
 - `--unfiltered_fasta` (large FASTA)
 - `--hmm_profiles` (HMMs for filtering).
- Output:
 - Candidate sequences (FASTA).

B. Alignment of Selected Sequences

- Tool: `MAFFT / HMMER`
- Input: Candidate sequences + `--reference_alignment`.
- Output: Aligned FASTA.

C. Phylogenetic Placement

- Same as Placement Mode (B).

D. Taxonomic Classification (Optional)

- Same as Placement Mode (C).

E. Reporting & Visualization

- Same as Placement Mode (D).

4. Nextflow Implementation Outline

```
nextflow
```

```
params {  
    // Input parameters  
    mode = 'placement'  
    queries = ''  
    unfiltered_fasta = ''  
    reference_alignment = ''  
    reference_tree = ''  
    model = 'GTR+G'  
    taxonomy = ''  
    hmm_profiles = ''  
}  
  
process input_validation {  
    // Check inputs based on mode  
}  
  
process sequence_search {  
    when: params.mode == 'search_placement'  
    // Run hmmsearch  
}  
  
process sequence_alignment {  
    // MAFFT/HMMER alignment  
}  
  
process phylogenetic_placement {  
    // EPA-ng/ppplacer
```



```

}

process taxonomic_classification {
  when: params.taxonomy
  // gappa/taxit
}

process reporting {
  // ETE3, R, Python for summaries
}

workflow {
  main:
    if (params.mode == 'placement') {
      input_validation(params.queries)
      sequence_alignment(params.queries)
    } else if (params.mode == 'search_placement') {
      input_validation(params.unfiltered_fasta)
      sequence_search(params.unfiltered_fasta, params.hmm_profiles)
      sequence_alignment(search_results)
    }
    phylogenetic_placement(aligned_sequences)
    taxonomic_classification(jplace_file)
    reporting()
}

```

5. Output Structure

```

results/
├─ aligned_sequences.fasta
├─ placements.jplace
├─ placed_tree.nwk
├─ taxonomy_assignments.tsv
├─ reports/
│   ├─ summary.html
│   └─ tree_visualization.pdf
└─ logs/

```

6. Recommended Tools & Profiles

- Containers: Docker/Singularity for reproducibility.
- Profiles:
 - `-profile docker` (pre-built images)
 - `-profile conda` (local tool installs)

Would you like additional details on any module (e.g., specific tool commands)?